PolyFlop8 Register File

Unit Test Card

Comprehensive Verification of Register Storage and Access

Document ID:	TC-REG-001
Revision:	1.1
Date:	January 6, 2026
Test Level:	Unit/Block Level
Test Engineer:	Artem Horiunov
Status:	DRAFT

Based on ISTQB Test Requirements Specification v1.0

1 REVISION HISTORY

1.0	Initial document creation	05.01.2026
1.1	Updated to reflect Rev 1.2 architectural change: addr_b changed from 3-bit to 8-bit with internal 3-bit slicing. Updated testbench signals, component declarations, and test stimuli.	06.01.2026

2 TEST OVERVIEW

2.1 Unit Under Test (UUT)

Component	Register File (8 registers, 8-bit each)
VHDL Entity	RegFile
Source File	Processor.txt (fourth entity)
Test Environment	ModelSim/QuestaSim 2023.4, VHDL Testbench
Number of Registers	8 (R0-R7)
Data Width	8-bit
Address Width	3-bit (internally sliced from addr_b[2:0])
Operation Type	Synchronous Write, Asynchronous Read
Clock Frequency	100MHz (10ns period)

2.2 Register File Architecture

Port	Width	Description
addr_a	3-bit	Read address for port A (direct 3-bit)
addr_b	8-bit	Read address for port B (8-bit input, lower 3 bits used for register indexing)
addr_w	3-bit	Write address
data_a, data_b	8-bit	Asynchronous read outputs
data_r7x	8-bit	Hardwired output of R7 (X-Pointer)
we	1-bit	Write enable (synchronous)
rf_src_sel	3-bit	Write data source multiplexer

2.3 Write Data Sources (rf_src_sel)

Code	Source	Description
"000"	alu_out	Result from ALU (arithmetic/logic operations)
"001"	ram_data	Data from memory (LD, POP instructions)
"010"	rs_imm_in	Immediate value (LDI, MOVI instructions)
"011"	io_data_in	Data from I/O ports (IN instruction)
"100"	sreg_data	Status register data (special operations)

2.4 Test Objectives

- Verify asynchronous read operation on both read ports
- Validate synchronous write operation with write enable

- Confirm dedicated R7 output (data_r7x) functionality
- Test all write data source multiplexer selections
- Verify register independence and data integrity
- Test reset functionality and initial state
- Verify 8-bit addr_b with internal 3-bit slicing works correctly

3 REQUIREMENTS COVERAGE

Req ID	Requirement Description	Test Case ID	Status
TR-REG-01	Verify asynchronous read on data_a and data_b based on addresses addr_a and addr_b	TC-REG-001-01	TBD
TR-REG-02	Verify synchronous write to register at addr_w only when we is high	TC-REG-001-02	TBD
TR-REG-03	Verify data_r7x acts as a dedicated hardwired output for Register 7	TC-REG-001-03	TBD
TR-REG-04	Verify Write Mux (rf_src_sel) correctly selects sources: ALU, RAM, Immediate, or I/O	TC-REG-001-04	TBD

Table 1: Requirements Coverage Matrix

4 DETAILED TEST CASES

4.1 Test Case TC-REG-001-01: Asynchronous Read Operation

Requirement: TR-REG-01
Purpose: Verify asynchronous read on data_a and data_b based on addresses addr_a and addr_b (including 8-bit addr_b with 3-bit slicing).

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
1.1	Read same register	Write R3 = 0x55 addr_a = "011" addr_b = x"03"	data_a = 0x55 data_b = 0x55
1.2	Read different registers	R1 = 0xAA, R2 = 0x55 addr_a = "001" addr_b = x"02"	data_a = 0xAA data_b = 0x55
1.3	Immediate address change	R0 = 0x11, R1 = 0x22 addr_a: "000" → "001"	data_a: 0x11 → 0x22
1.4	Read all registers	Initialize all registers Scan addr_a from 0 to 7	data_a shows each value
1.5	Read during write	R4 = 0x33 (old) Write R4 = 0x44 Read R4 during write	data_a = 0x33
1.6	Read after write	Write R5 = 0x66 Wait one clock cycle Read R5	data_a = 0x66

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
1.7	Port independence	R6 = 0x77, R7 = 0x88 addr_a = "110" addr_b = x"07" Change only addr_a	data_a changes, data_b remains 0x88
1.8	Read undefined register	After reset Read any register	data_a = 0x00 data_b = 0x00
1.9	Addr_b upper bits ignored	R3 = 0x5A addr_b = x"83" (bit 7=1) Lower 3 bits = "011"	data_b = 0x5A (uses only bits 2:0)
1.10	8-bit addr_b edge cases	Set R0=0x01, R7=0x02 addr_b = x"07" Then addr_b = x"00"	data_b: 0x02 → 0x01 (uses lower 3 bits only)

4.2 Test Case TC-REG-001-03: Dedicated R7 Output

Requirement: TR-REG-03
Purpose: Verify data_r7x acts as a dedicated hardwired output for Register 7.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
3.1	Direct R7 write and read	Write R7 = 0x5A we = '1' addr_w = "111"	data_r7x = 0x5A
3.2	data_r7x independence	R7 = 0x77 Change addr_a, addr_b	data_r7x = 0x77
3.3	Continuous R7 monitoring	Write sequence to R7: 0x11, 0x22, 0x33	data_r7x: 0x11 → 0x22 → 0x33
3.4	data_r7x vs data_b	R7 = 0x88 Read R7 via addr_b = x"07"	data_b = 0x88 data_r7x = 0x88
3.5	R7 as X-Pointer	R7 = 0xF0 Use data_r7x as memory address	data_r7x = 0xF0
3.6	R7 reset value	After system reset	data_r7x = 0x00
3.7	R7 write independence	R6 = 0x66, R7 = 0x77 Write R7 = 0x99	R6 remains 0x66 data_r7x = 0x99

5 TEST BENCH ARCHITECTURE

5.1 VHDL Testbench Structure (Updated for 8-bit addr_b)

```
-- Register File Testbench (regfile_tb.vhd)
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity regfile_tb is
end regfile_tb;

architecture Behavioral of regfile_tb is
    -- Component Declaration (UPDATED for 8-bit addr_b)
    component RegFile is
        Port (
```

```
            clk           : in  STD_LOGIC;
            reset         : in  STD_LOGIC;
            we            : in  STD_LOGIC;
            addr_a        : in  STD_LOGIC_VECTOR (2 downto 0);
            addr_b        : in  STD_LOGIC_VECTOR (7 downto 0);  -- Changed to 8-bit
            addr_w        : in  STD_LOGIC_VECTOR (2 downto 0);
            rf_src_sel    : in  STD_LOGIC_VECTOR (2 downto 0);
            alu_out       : in  STD_LOGIC_VECTOR (7 downto 0);
            ram_data      : in  STD_LOGIC_VECTOR (7 downto 0);
            rs_imm_in     : in  STD_LOGIC_VECTOR (7 downto 0);
            io_data_in    : in  STD_LOGIC_VECTOR (7 downto 0);
            sreg_data     : in  STD_LOGIC_VECTOR (7 downto 0);
            data_a        : out STD_LOGIC_VECTOR (7 downto 0);
            data_b        : out STD_LOGIC_VECTOR (7 downto 0);
            data_r7x      : out STD_LOGIC_VECTOR (7 downto 0)
        );
    end component;

    -- Test Signals (UPDATED addr_b_tb to 8-bit)
    signal clk_tb        : STD_LOGIC := '0';
    signal reset_tb      : STD_LOGIC := '0';
    signal we_tb         : STD_LOGIC := '0';
    signal addr_a_tb     : STD_LOGIC_VECTOR (2 downto 0) := (others => '0');
    signal addr_b_tb     : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');  -- 8-bit
    signal addr_w_tb     : STD_LOGIC_VECTOR (2 downto 0) := (others => '0');
    signal rf_src_sel_tb : STD_LOGIC_VECTOR (2 downto 0) := (others => '0');
    signal alu_out_tb    : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
    signal ram_data_tb   : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
    signal rs_imm_in_tb  : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
    signal io_data_in_tb : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
    signal sreg_data_tb  : STD_LOGIC_VECTOR (7 downto 0) := (others => '0');
    signal data_a_tb     : STD_LOGIC_VECTOR (7 downto 0);
    signal data_b_tb     : STD_LOGIC_VECTOR (7 downto 0);
    signal data_r7x_tb   : STD_LOGIC_VECTOR (7 downto 0);

    -- Clock period
    constant CLK_PERIOD : time := 10 ns;

    -- Test control
    signal test_phase : integer := 0;

    -- Internal register array for reference checking
    type reg_array is array (0 to 7) of std_logic_vector(7 downto 0);
    signal expected_registers : reg_array := (others => (others => '0'));

begin
    -- Unit Under Test Instantiation
    UUT: RegFile port map (
        clk           => clk_tb,
        reset         => reset_tb,
        we            => we_tb,
        addr_a        => addr_a_tb,
        addr_b        => addr_b_tb,
        addr_w        => addr_w_tb,
        rf_src_sel    => rf_src_sel_tb,
        alu_out       => alu_out_tb,
        ram_data      => ram_data_tb,
        rs_imm_in     => rs_imm_in_tb,
        io_data_in    => io_data_in_tb,
        sreg_data     => sreg_data_tb,
        data_a        => data_a_tb,
        data_b        => data_b_tb,
        data_r7x      => data_r7x_tb
```

```
    );

    -- Clock Generation Process
    clk_process: process
    begin
        clk_tb <= '0';
        wait for CLK_PERIOD/2;
        clk_tb <= '1';
        wait for CLK_PERIOD/2;
    end process;

    -- Main Test Stimulus Process (UPDATED for 8-bit addr_b)
    stim_proc: process
        procedure check_register(
            reg_addr : in integer;
            expected : in std_logic_vector(7 downto 0);
            message  : in string
        ) is
        begin
            addr_a_tb <= std_logic_vector(to_unsigned(reg_addr, 3));
            wait for 1 ns; -- Allow async read to settle
            assert data_a_tb = expected
                report message & ": Register R" & integer'image(reg_addr) &
                       " expected 0x" & to_hstring(expected) &
                       ", got 0x" & to_hstring(data_a_tb)
                severity error;
        end procedure;

    begin
        report "Starting Register File Test Bench (Rev 1.2: 8-bit addr_b)";

        -- Initialize
        reset_tb <= '1';
        wait for CLK_PERIOD;
        reset_tb <= '0';
        wait for CLK_PERIOD;

        -- Test Case 1.1: Asynchronous read same register (UPDATED)
        report "Test Case 1.1: Asynchronous read same register";
        -- First write a value to R3
        we_tb <= '1';
        addr_w_tb <= "011";
        rf_src_sel_tb <= "010"; -- Immediate source
        rs_imm_in_tb <= x"55";
        wait until rising_edge(clk_tb);
        wait for 1 ns;

        -- Now read R3 on both ports (addr_b uses 8-bit)
        we_tb <= '0';
        addr_a_tb <= "011";
        addr_b_tb <= x"03";  -- 8-bit address for R3
        wait for 1 ns;
        assert data_a_tb = x"55" and data_b_tb = x"55"
            report "Read same register failed" severity error;

        -- Test Case 1.9: Addr_b upper bits ignored
        report "Test Case 1.9: Addr_b upper bits ignored";
        -- Test with upper bits set
        addr_b_tb <= x"83";  -- binary: 10000011, lower 3 bits = 011 (R3)
        wait for 1 ns;
        assert data_b_tb = x"55"
            report "addr_b upper bits not ignored: Expected 0x55, got 0x" &
                   to_hstring(data_b_tb)
```

```
            severity error;

        -- Continue with other test cases...

        -- Final Report
        report "All Register File test cases completed successfully!";
        wait;
    end process;

    -- Monitor process for debugging
    monitor_proc: process
    begin
        wait until rising_edge(clk_tb);
        wait for 1 ns; -- Allow signals to propagate
        report "Time: " & time'image(now) &
               " | data_a: 0x" & to_hstring(data_a_tb) &
               " | data_b: 0x" & to_hstring(data_b_tb) &
               " | data_r7x: 0x" & to_hstring(data_r7x_tb) &
               " | addr_b: 0x" & to_hstring(addr_b_tb) &
               " | we: " & std_logic'image(we_tb);
    end process;

end Behavioral;
```

6 SPECIAL TEST CONSIDERATIONS

6.1 8-bit addr_b with 3-bit Slicing

- Verify only lower 3 bits of addr_b are used for register addressing
- Test with upper 5 bits set to various values to ensure they are ignored
- Verify addr_b changes are immediately reflected on data_b (asynchronous)
- Test boundary conditions: addr_b = x"00" to x"07" (valid addresses)
- Test addr_b values where lower 3 bits wrap around: x"08" (should access R0)

6.2 Timing Considerations

- Verify asynchronous read delay <1ns (combinational path)
- Test setup/hold times for write address and data
- Verify clock-to-output delay for synchronous behavior
- Test recovery from timing violations
- Ensure addr_b 8-bit bus meets timing requirements